EXPERIMENT 5

Student Name: Arnav Sharma **UID: 24BAI70781**
Branch: AIT-CSE **Section/Group: 24AIT_KRG-G2**
Semester: 4
Subject Name: Database Management System **Subject Code: 24CSH-298**

Experiment 5 – SQL Conditional Logic (Odd & Even Values)

Experiment 5: SQL Conditional Logic using the MOD (%) operator to classify employee salary values as Odd or Even. This experiment demonstrates numerical analysis and conditional evaluation using SQL queries in Oracle and PostgreSQL.

Aim
To understand and apply conditional logic in SQL by using the modulus operator (MOD / %) to analyze numerical data and classify employee salaries as odd or even, thereby improving data analysis and decision-making skills in SQL.

Objective
• To write and execute SQL queries using the MOD (%) operator.
• To retrieve salary details from an employee table.
• To classify salaries into Odd and Even categories.
• To display odd and even salary values separately.

Software Requirements

Database Management System:
Oracle Database Express Edition (Oracle XE)
PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

Practical / Experiment Steps

1. Create an employee table with columns:
   id
   name

department
salary

2. Insert sample employee salary records into the table.
3. Write a SQL query to retrieve employee salary details.
4. Use the MOD(salary, 2) function (Oracle) or salary % 2 (PostgreSQL) to check whether the salary is odd or even.
5. Use a CASE statement to classify salaries as:
   'Odd Salary'
   'Even Salary'
6. Execute separate queries to:
   Display only employees with Odd salaries.
   Display only employees with Even salaries.
7. Observe and record the output results.

Procedure

1. Open Oracle SQL Developer or pgAdmin.
2. Connect to the database.
3. Create the employee table using CREATE TABLE.
4. Insert sample records using INSERT INTO.
5. Execute SELECT queries with the MOD operator.
6. Apply CASE statement for conditional classification.
7. Display results and capture screenshots.

Input / Output Details

Input

Table Name: employee

Columns:
id (Integer)
name (Varchar)
department (Varchar)
salary (Integer)

EXPERIMENT 5

Logic Used:
MOD(salary, 2) or salary % 2
CASE statement

Step-wise Output

Step 1 – Create employee Table

```
CREATE TABLE employee (
    id int PRIMARY KEY,
    name       VARCHAR(50),
    department VARCHAR(50),
    salary     INT
);
```

Step 2 – Insert Data into employee Table

```
INSERT INTO employee VALUES (1, 'Alice',   'HR',        55000);
INSERT INTO employee VALUES (2, 'Bob',     'Finance',   62000);
INSERT INTO employee VALUES (3, 'Charlie', 'IT',        47001);
INSERT INTO employee VALUES (4, 'Diana',   'Marketing', 38500);
INSERT INTO employee VALUES (5, 'Edward',  'IT',        73001);

SELECT * FROM employee;
```

Step 3 – Display Employee Salary Records

```
SELECT * FROM employee;
SELECT id, name, department, salary
FROM employee;
```

EXPERIMENT 5

Step 4 – Classify Salaries as Odd or Even

```
SELECT id, name, salary,
    CASE
        WHEN MOD(salary, 2) = 0 THEN 'Even Salary'
        ELSE 'Odd Salary'
    END AS salary_type
FROM employee;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name	salary	salary_type
1	Alice	55000	Even Salary
2	Bob	62000	Even Salary
3	Charlie	47001	Odd Salary
4	Diana	38500	Even Salary
5	Edward	73001	Odd Salary

Step 5 – Display Only Odd Salaries

```
SELECT id, name, department, salary
FROM employee
WHERE MOD(salary, 2) <> 0;
```

Result Grid | Filter Rows: | Edit: | Ex

id	name	department	salary
3	Charlie	IT	47001
5	Edward	IT	73001
NULL	NULL	NULL	NULL

EXPERIMENT 5

Step 6 – Display Only Even Salaries

```
SELECT id, name, department, salary
FROM employee
WHERE MOD(salary, 2) = 0;
```

Result Grid | Filter Rows: | Edit: | Export/

id	name	department	salary
1	Alice	HR	55000
2	Bob	Finance	62000
4	Diana	Marketing	38500
NULL	NULL	NULL	NULL

Learning Outcome

After completing this experiment, the learner will be able to:

- Understand conditional logic in SQL queries.
- Use the MOD (%) operator for numerical analysis.
- Classify numeric values into odd and even categories.
- Write efficient SQL queries for payroll data analysis.
- Apply SQL logic in real-world enterprise environments.